

```
!pip install transformers torch
```

```
Requirement already satisfied: transformers in /usr/local/lib/python3.12/dist-packages (4.57.1)
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.8.0+cu126)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from transformers) (3.20.0)
Requirement already satisfied: huggingface-hub<1.0,>=0.34.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.27.0)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (25.0)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from transformers) (6.0.3)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2024.11.6)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from transformers) (2.32.4)
Requirement already satisfied: tokenizers<0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.20.3)
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.6.2)
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.12/dist-packages (from transformers) (4.67.1)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.13.3)
Requirement already satisfied: networkx in /usr/local/lib/python3.12/dist-packages (from torch) (3.5)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec in /usr/local/lib/python3.12/dist-packages (from torch) (2025.3.0)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch) (12.6.77)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch) (12.6.77)
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.6.80 in /usr/local/lib/python3.12/dist-packages (from torch) (12.6.80)
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from torch) (9.10.2.21)
Requirement already satisfied: nvidia-cublas-cu12==12.6.4.1 in /usr/local/lib/python3.12/dist-packages (from torch) (12.6.4.1)
Requirement already satisfied: nvidia-cufft-cu12==11.3.0.4 in /usr/local/lib/python3.12/dist-packages (from torch) (11.3.0.4)
Requirement already satisfied: nvidia-curand-cu12==10.3.7.77 in /usr/local/lib/python3.12/dist-packages (from torch) (10.3.7.77)
Requirement already satisfied: nvidia-cusolver-cu12==11.7.1.2 in /usr/local/lib/python3.12/dist-packages (from torch) (11.7.1.2)
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch) (0.7.1)
Requirement already satisfied: nvidia-nccl-cu12==2.27.3 in /usr/local/lib/python3.12/dist-packages (from torch) (2.27.3)
Requirement already satisfied: nvidia-cusparse-cu12==12.5.4.2 in /usr/local/lib/python3.12/dist-packages (from torch) (12.5.4.2)
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch) (0.7.1)
Requirement already satisfied: nvidia-nvtx-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch) (12.6.77)
Requirement already satisfied: nvidia-nvjitlink-cu12==12.6.85 in /usr/local/lib/python3.12/dist-packages (from torch) (12.6.85)
Requirement already satisfied: nvidia-cufile-cu12==1.11.1.6 in /usr/local/lib/python3.12/dist-packages (from torch) (1.11.1.6)
Requirement already satisfied: triton==3.4.0 in /usr/local/lib/python3.12/dist-packages (from torch) (3.4.0)
Requirement already satisfied: hf-xet<2.0.0,>=1.1.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<1.0,>=0.34.0->transformers) (1.1.3)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch) (1.37.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2->torch) (3.0.3)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->transformers) (3.4.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->transformers) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->transformers) (2.31.2)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->transformers) (2025.11.12)
```

✧ Bài 1: Khởi phục Masked Token (Masked Language Modeling)

```
from transformers import pipeline
# 1. Tải pipeline "fill-mask"
# Pipeline này sẽ tự động tải một mô hình mặc định phù hợp (thường là một biến thể của BERT)
mask_filler = pipeline("fill-mask")
# 2. Câu đầu vào với token <mask>
input_sentence = "Hanoi is the <mask> of Vietnam."
# 3. Thực hiện dự đoán
# top_k=5 yêu cầu mô hình trả về 5 dự đoán hàng đầu
predictions = mask_filler(input_sentence, top_k=5)
# 4. In kết quả
print(f"Câu gốc: {input_sentence}")
for pred in predictions:
    print(f"Dự đoán: '{pred['token_str']}' với độ tin cậy: {pred['score']:.4f}")
    print(f"-> Câu hoàn chỉnh: {pred['sequence']}")
```

```
No model was supplied, defaulted to distilbert/distilroberta-base and revision fb53ab8 (https://huggingface.co/distilbert/distilroberta-base)
Using a pipeline without specifying a model name and revision in production is not recommended.
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as the secret `HF_TOKEN` in your Colab secrets, and restart this notebook.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(

config.json: 100%                               480/480 [00:00<00:00, 27.6kB/s]

model.safetensors: 100%                         331M/331M [00:02<00:00, 240MB/s]

Some weights of the model checkpoint at distilbert/distilroberta-base were not used when initializing RobertaForMaskedLM: ['
- This IS expected if you are initializing RobertaForMaskedLM from the checkpoint of a model trained on another task or with
- This IS NOT expected if you are initializing RobertaForMaskedLM from the checkpoint of a model that you expect to be exact

tokenizer_config.json: 100%                      25.0/25.0 [00:00<00:00, 3.10kB/s]

vocab.json:      899k/? [00:00<00:00, 23.7MB/s]

merges.txt:      456k/? [00:00<00:00, 29.0MB/s]

tokenizer.json:   1.36M/? [00:00<00:00, 48.6MB/s]

Device set to use cuda:0
Câu gốc: Hanoi is the <mask> of Vietnam.
Dự đoán: ' capital' với độ tin cậy: 0.9341
-> Câu hoàn chỉnh: Hanoi is the capital of Vietnam.
Dự đoán: ' Republic' với độ tin cậy: 0.0300
-> Câu hoàn chỉnh: Hanoi is the Republic of Vietnam.
Dự đoán: ' Capital' với độ tin cậy: 0.0105
-> Câu hoàn chỉnh: Hanoi is the Capital of Vietnam.
Dự đoán: ' birthplace' với độ tin cậy: 0.0054
-> Câu hoàn chỉnh: Hanoi is the birthplace of Vietnam.
Dự đoán: ' heart' với độ tin cậy: 0.0014
-> Câu hoàn chỉnh: Hanoi is the heart of Vietnam.
```

✧ Bài 2: Dự đoán từ tiếp theo (Next Token Prediction)

```
from transformers import pipeline
# 1. Tải pipeline "text-generation"
# Pipeline này sẽ tự động tải một mô hình phù hợp (thường là GPT-2)
generator = pipeline("text-generation")
# 2. Đoạn văn bản mẫu
prompt = "The best thing about learning NLP is"
# 3. Sinh văn bản
# max_length: tổng độ dài của câu mẫu và phần được sinh ra
# num_return_sequences: số lượng chuỗi kết quả muốn nhận
generated_texts = generator(prompt, max_length=50, num_return_sequences=1)
# 4. In kết quả
print(f"Câu mẫu: '{prompt}'")
for text in generated_texts:
    print("Văn bản được sinh ra:")
    print(text['generated_text'])
```

cation strategy. If you encode pairs of sequences (GLUE-style) with the tokenizer you can select this strategy more precisely by passing 'pairwise' as the `strategy` argument. For more details, see [this page](https://huggingface.co/transformers/main/en/main_classes/text_generation).

ty are very smart and creative.

to learn all the techniques that I would have worked on in my second semester. How can you get this? No, you aren't going to benefit of the NLP?

back to law school, or you have to go to college. So I think you can get back into the NLP if you're going to

✧ Bài 3: Tính toán Vector biểu diễn của câu (Sentence Representation)

```
import torch
from transformers import AutoTokenizer, AutoModel

# 1. Chọn một mô hình BERT
model_name = "bert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModel.from_pretrained(model_name)
```

```
# 2. Câu đầu vào
sentences = ["This is a sample sentence."]
# 3. Tokenize câu
# padding=True: đệm các câu ngắn hơn để có cùng độ dài
# truncation=True: cắt các câu dài hơn
# return_tensors='pt': trả về kết quả dưới dạng PyTorch tensors
inputs = tokenizer(sentences, padding=True, truncation=True, return_tensors='pt')
# 4. Đưa qua mô hình để lấy hidden states
# torch.no_grad() để không tính toán gradient, tiết kiệm bộ nhớ
with torch.no_grad():
    outputs = model(**inputs)
# outputs.last_hidden_state chứa vector đầu ra của tất cả các token
last_hidden_state = outputs.last_hidden_state
# shape: (batch_size, sequence_length, hidden_size)
# 5. Thực hiện Mean Pooling
# Để tính trung bình chính xác, chúng ta cần bỏ qua các token đệm (padding tokens)
attention_mask = inputs['attention_mask']
mask_expanded = attention_mask.unsqueeze(-1).expand(last_hidden_state.size()).float()
sum_embeddings = torch.sum(last_hidden_state * mask_expanded, 1)
sum_mask = torch.clamp(mask_expanded.sum(1), min=1e-9)
sentence_embedding = sum_embeddings / sum_mask
# 6. In kết quả
print("Vector biểu diễn của câu:")
print(sentence_embedding)
print("\nKích thước của vector:", sentence_embedding.shape)
```



```
tokenizer_config.json: 100%          48.0/48.0 [00:00<00:00, 4.00kB/s]
config.json: 100%                    570/570 [00:00<00:00, 73.6kB/s]
vocab.txt: 100%                      232k/232k [00:00<00:00, 3.57MB/s]
tokenizer.json: 100%                 466k/466k [00:00<00:00, 7.36MB/s]
model.safetensors: 100%             440M/440M [00:04<00:00, 111MB/s]
Vector biểu diễn của câu:
tensor([[ -6.3875e-02, -4.2837e-01, -6.6779e-02, -3.8430e-01, -6.5785e-02,
          -2.1826e-01,  4.7636e-01,  4.8659e-01,  3.9689e-05, -7.4274e-02,
          -7.4740e-02, -4.7635e-01, -1.9773e-01,  2.4824e-01, -1.2162e-01,
           1.6678e-01,  2.1045e-01, -1.4576e-01,  1.2637e-01,  1.8635e-02,
           2.4640e-01,  5.7090e-01, -4.7014e-01,  1.3782e-01,  7.3650e-01,
          -3.3808e-01, -5.0329e-02, -1.6453e-01, -4.3517e-01, -1.2900e-01,
           1.6516e-01,  3.4004e-01, -1.4930e-01,  2.2422e-02, -1.0488e-01,
          -5.1916e-01,  3.2964e-01, -2.2162e-01, -3.4206e-01,  1.1993e-01,
          -7.0148e-01, -2.3126e-01,  1.1224e-01,  1.2550e-01, -2.5191e-01,
          -4.6374e-01, -2.7261e-02, -2.8415e-01, -9.9250e-02, -3.7018e-02,
          -8.9192e-01,  2.5005e-01,  1.5816e-01,  2.2701e-01, -2.8497e-01,
           4.5300e-01,  5.0921e-03, -7.9441e-01, -3.1008e-01, -1.7403e-01,
```